# MYSX-256/512: A 256-bit Block Cipher with 512-bit Keys (v0.1-draft)

StringManolo

2026

**Abstract**

We present MYSX-256/512, a 256-bit block cipher with 512-bit keys and 128 rounds. The design combines Feistel cells with Simon's round function, a Myszkowski byte transposition, word rotations, and an ARX key schedule. We provide the full specification, test vectors, and a self-analysis covering differential, linear, and related-key cryptanalysis. The differential analysis reports an observed upper bound on the best single-trail weight (114 at 14 rounds, extrapolating to the 128-round threshold at r≈15). The r=4 trail has been directly verified at w=32 with a clustering factor of 2.00× (observed $2^{-7}$ vs MILP $2^{-8}$). The linear analysis attempted three models; none produced trails that verified empirically at w=32, and a scaled-down (w=8) experiment with a hand-validated harness confirms no detectable linear signal for low-weight α masks. The key schedule was audited with 22 structural and empirical tests up to weight-6 with no persistent Δrk=0 pattern found. MYSX has not been externally audited. It is released as v0.1-draft for public review and must not be used to protect real data.

## 1. Introduction

MYSX-256/512 is an experimental block cipher. It is offered as a public proposal for cryptanalysis, not as a production primitive. The author makes no claim of security beyond the observed results reported here, and explicitly welcomes external review.

### 1.1. Security objectives (and non-objectives)

**Model.** MYSX is analyzed in the single-key, chosen-plaintext model for confidentiality, combined with an encrypt-then-MAC AEAD wrapper for integrity (Section 2.9). The design does not claim security under related-key, chosen-ciphertext, or side-channel models; see Section 5 for related-key coverage and Section 9 for engineering notes.

**Claimed security level.** None formally. The paper reports observed upper bounds on single-trail weights (Section 3) and negative results on three linear-trail models (Section 4). These are inputs to cryptanalysis, not proofs.

**Non-objectives.** MYSX does not claim:

- post-quantum security beyond the trivial Grover bound ($2^{256}$ quantum iterations for a 512-bit key);

- resistance to related-key attacks beyond the heuristic coverage in Section 5;
- resistance to integral, impossible-differential, boomerang, or algebraic attacks (none analyzed);
- constant-time execution in JavaScript (side channels out of scope).

## 1.2. Design rationale

MYSX combines four building blocks:

- **Simon's F** `F(x) = (rotl(x,1) ∧ rotl(x,8)) ⊕ rotl(x,2)`. Chosen for its **studied** diffusion/linear/differential properties in Simon and its simplicity (no S-boxes, only AND/XOR/rotl). It does **not** inherit Simon's security proofs; the surrounding structure is different. See Section 7 for what transfers and what does not.
- **Feistel cells over 32-bit word pairs.** Four parallel cells per round, then a byte transposition over the full 32-byte state, then word rotations. The result is classified as an SPN-like structure with Feistel cells, not as a "2-branch Feistel". See Section 2.1.
- **Myszkowski transposition** (Section 2.3.1). A fixed byte permutation derived from the keyword `MYSZKOWS`. The choice is didactic; it provides byte-level diffusion but no cryptographic argument beyond that. An optimized permutation would provide strictly better diffusion per round; see Section 9.
- **ARX key schedule** (Section 2.4). A bijective Feistel 8+8 update with modular addition, XOR, and rotations. Chosen for simplicity and to avoid S-boxes; analyzed in Section 5.

The parameter choices (256-bit block, 512-bit key, 128 rounds) are not derived from a formal argument. They were chosen to:

- provide a large block (avoid birthday-bound issues of 64-bit blocks);
- provide a large key (Grover leaves 256-bit security);
- provide a large round count relative to Simon128/128 (68 rounds) as a precaution against unknown trails.

No claim is made that 128 rounds is sufficient. The round count is a parameter, not a security level.

## 1.3. Comparison with related designs

| Design | Block | Key | Rounds | Structure | S-boxes |
|---|---|---|---|---|---|
| Simon128/128 | 128 | 128 | 68 | Feistel | No |
| Speck128/256 | 128 | 256 | 34 | ARX | No |
| ChaCha20 | 512 | 256 | 20 | ARX | No |
| MYSX-256/512 | 256 | 512 | 128 | SPN-like with Feistel cells | No |

MYSX's closest sibling is Simon. The differences (block size, branch organization, transposition, word rotations, non-linear key schedule) are substantial enough that Simon cryptanalysis does not transfer directly (see Section 7).

**Performance.** MYSX is estimated to run at a few MB/s in JavaScript, versus hundreds of MB/s for AES-GCM via WebCrypto (AES-NI). MYSX is not competitive on performance; it is a research primitive.

# 2. Specification

## 2.1. Parameters

| Parameter | Value |
|---|---|
| Block size | 256 bits (8 words of 32 bits) |
| Key size | 512 bits (16 words of 32 bits) |
| Rounds | 128 |
| Structure | SPN-like: 4 parallel Feistel cells + byte transposition |
| Non-linearity | `F(x) = (rotl(x,1) ∧ rotl(x,8)) ⊕ rotl(x,2)` |
| Diffusion | Myszkowski transposition (keyword `MYSZKOWS`) |
| Rotation | `rotl(word, (r mod 31) + 1)` per word and round |
| Final whitening | XOR with 8 words from the key schedule |

**Structural note.** MYSX is sometimes described as a "2-branch Feistel" by analogy with Simon. Strictly, each round applies four independent Feistel cells over 32-bit word pairs, then a byte transposition over the full 32-byte state, then word rotations. It is more accurately classified as an SPN with Feistel cells. The "Feistel" label is a convenience.

## 2.2. Notation

- `rotl(x, n)`: circular left rotation of `x` by `n` bits.
- `∧`: bitwise AND.
- `⊕`: bitwise XOR.
- `s[r]`: state at round `r` (`s[0]` = plaintext, `s[128]` = ciphertext).
- `s[r][i]`: word `i` (0..7) of the state at round `r`.
- `rk[r][i]`: subkey `i` (0..7) of round `r`.
- `rkFinal[i]`: word `i` (0..7) of the final whitening.

## 2.3. Constants

```
CONST[0..7] = [
  0x243f6a88, 0x85a308d3, 0x13198a2e, 0x03707344,
  0xa4093822, 0x299f31d0, 0x082efa98, 0xec4e6c89
]
```

These are the fractional digits of π (same as Blowfish).

### 2.3.1. Myszkowski permutation

Keyword: `MYSZKOWS` (8 letters, S repeated at positions 3 and 8).

Alphabetical order: K(5), M(1), O(6), S(3,8), W(7), Y(2), Z(4).

The byte permutation `MYZ_PERM[0..31]` is constructed as:

```
MYZ_PERM = [
   4,12,20,28,                 # K (column 4)
   0, 8,16,24,                 # M (column 0)
   5,13,21,29,                 # O (column 5)
   2, 7,10,15,18,23,26,31,     # S,S (columns 2 and 7, read row-wise)
   6,14,22,30,                 # W (column 6)
   1, 9,17,25,                 # Y (column 1)
   3,11,19,27                  # Z (column 3)
]
```

The inverse permutation `MYZ_INV` satisfies `MYZ_INV[MYZ_PERM[i]] = i`.

**Design note.** The Myszkowski choice is didactic (classical transposition with a keyword) and adds no cryptographic argument. A byte permutation optimized for diffusion (e.g., a structured permutation over the 4×8 byte grid) would strictly dominate it. We keep Myszkowski in v0.1 because it is the design under review; a future version may replace it.

## 2.4. Key schedule

### 2.4.1. Initialization

Input: key `K[0..63]` (64 bytes).

Initial state `s[0..15]` (16 words of 32 bits):

```
s[i] = big_endian_word(K[4i .. 4i+3])
```

### 2.4.2. Expansion

For `r = 0 .. 127`:

```
spread = (r + 1) * 0x9E3779B9  (mod 2^32)

# Round subkeys
for i in 0..7:
    rk[r][i] = s[i] ⊕ s[(i+5) mod 16] ⊕ s[(i+11) mod 16]
               ⊕ CONST[(r+i) mod 8] ⊕ spread

# For i = 0..3: fold fk into rk
for i in 0..3:
    fk = s[(i+8) mod 16] ⊕ CONST[(r+i+3) mod 8]
    rk[r][i] = rk[r][i] ⊕ fk

# State update (Feistel 8+8, bijective)
L = s[0..7]
R = s[8..15]
```

```
for i in 0..7:
    a = R[i]
    b = R[(i+1) mod 8]
    c = R[(i+3) mod 8]
    F = (rotl((a + b) mod 2^32, 7) ⊕ rotl(c, 13) ⊕ CONST[i] ⊕ spread)
    s[i]     = R[i]
    s[i + 8] = L[i] ⊕ F
```

#### 2.4.3. Final whitening

After the 128 rounds:

```
for i in 0..7:
    rkFinal[i] = s[i] ⊕ s[(i+8) mod 16] ⊕ CONST[(128+i) mod 8]
```

### 2.5. Round function

Each round `r` transforms state `s[r]` into `s[r+1]` in four steps:

**Step 1. AddRoundKey.**

```
for i in 0..7:
    s[r][i] = s[r][i] ⊕ rk[r][i]
```

**Step 2. Feistel cells.** For `p = 0..3`:

```
L = s[r][p]
R = s[r][p + 4]
s_mid[p]     = R
s_mid[p + 4] = L ⊕ F(R)
```

where `F(x) = (rotl(x,1) ∧ rotl(x,8)) ⊕ rotl(x,2)`.

**Step 3. Myszkowski transposition.** Convert `s_mid[0..7]` to bytes (big-endian per word):

```
bytes[4i .. 4i+3] = big_endian(s_mid[i])
permuted[j] = bytes[MYZ_PERM[j]]  for j = 0..31
s_perm[i] = big_endian_word(permuted[4i .. 4i+3])
```

**Step 4. Word rotation.**

```
shift = (r mod 31) + 1
for i in 0..7:
    s[r+1][i] = rotl(s_perm[i], shift)
```

### 2.6. Encryption

```
s[0] = plaintext  (as 8 big-endian words)
for r = 0..127:
    apply round r (Section 2.5)
for i = 0..7:
    s[128][i] = s[128][i] ⊕ rkFinal[i]
```

```
ciphertext = s[128]  (as 32 big-endian bytes)
```

### 2.7. Decryption

Inverse of encryption:

```
s[128] = ciphertext
for i = 0..7:
    s[128][i] = s[128][i] ⊕ rkFinal[i]
for r = 127..0:
    apply inverse round r (inverse of Section 2.5, reverse order)
plaintext = s[0]
```

The inverse round consists of:

1. Inverse rotation: `rotl(word, 32 - shift)`.
2. Inverse transposition: `bytes[MYZ_PERM[j]] = permuted[j]`.
3. Inverse Feistel: `R = newL`, `L = newR ⊕ F(R)`.
4. XOR with `rk[r][i]`.

### 2.8. Mode of operation (CTR)

32-byte counter:

```
counter[0..15]  = nonce (16 bytes)
counter[16..31] = 0  (128 bits, big-endian)
```

For each 32-byte plaintext block:

```
keystream = MYSX_encrypt(counter)
ciphertext_block = plaintext_block ⊕ keystream
counter[16..31] += 1  (big-endian)
```

### 2.9. AEAD (encrypt-then-MAC)

Packet: `VERSION(1) ‖ NONCE(16) ‖ CT(n) ‖ TAG(32)`

MAC input: `VERSION ‖ NONCE ‖ AAD_LEN(4 BE) ‖ AAD ‖ CT_LEN(4 BE) ‖ CT`

TAG: `HMAC-SHA256(macKey, MAC input)`

Verification: the decryptor verifies the TAG before decrypting.

### 2.10. Key derivation

**From a random secret (HKDF-SHA256).**

```
encKey = HKDF-SHA256(secret, salt, info = context ‖ ":enc", L = 64)
macKey = HKDF-SHA256(secret, salt, info = context ‖ ":mac", L = 32)
```

**From a password (PBKDF2-SHA256 + HKDF).**

```
prk    = PBKDF2-SHA256(password, salt, iterations = 600000, L = 32)
encKey = HKDF-SHA256(prk, salt, info = context ‖ ":enc", L = 64)
macKey = HKDF-SHA256(prk, salt, info = context ‖ ":mac", L = 32)
```

### 2.11. Test vectors

See `KAT.json` (public). Two digests are documented in `README.md`:

- `KAT.json` public digest: SHA-256 over the full JSON (block + AEAD + KDF). This is the artifact a third-party implementation should reproduce.
- `KNOWN_KAT_DIGEST` (internal): SHA-256 over 16 blocks with `key[i] = i`. Used as fast regression in `mysx.test.js`. It is strictly a subset of the public KAT, not a different KAT.

A third-party implementation should target the **public digest**.

## 3. Differential cryptanalysis

### 3.1. Model

Tool: `mysx_milp.py` (PuLP + CBC/HiGHS). AND model: `a = Δx ∨ Δy`, which counts a cost-1 event whenever at least one input is active. For an AND with independent inputs, this is the exact per-bit cost. The real approximations in the model are:

- The Markov-independence assumption does not hold: the two rotated inputs of F come from the same word `x`.
- We ran a single input difference at r=12–14 (w2b30). Multi-bit input differences may yield lower weights.
- `gapRel=0.2` and `timeLimit=300` return feasible solutions (upper bounds), not certified optima.
- The model captures single trails, not differentials (clusters of trails with the same output difference).

### 3.2. Observed upper bounds

| Rounds | Input | Weight (upper bound) | Single-trail prob. |
|---|---|---|---|
| 5 | w1b24 | 2 | $2^{-2}$ |
| 6 | w1b24 | 6 | $2^{-6}$ |
| 7 | w2b30 | 10 | $2^{-10}$ |
| 8 | w2b30 | 18 | $2^{-18}$ |
| 12 | w2b30 | 68 | $2^{-68}$ |
| 14 | w2b30 | 114 | $2^{-114}$ |

Extrapolation: the 128-bit threshold is reached at r≈15. **This is an extrapolation, not a proof.** It tells us what the model produces for the input we tried; it does not establish the best trail over all inputs, nor that clustering does not lower the effective security.

**Direct verification at r=4 (w=32).** The trail from the table above (w0b0, MILP weight 8, Δout = {w0b4, w1b10, w1b16, w1b30, w6b4}) was verified with 8 fixed keys × $2^{17}$ pairs each, using the full 128-round cipher with the final whitening disabled (to isolate the round function). Observed hit rate: $7.802 \times 10^{-3}$ over 1,048,576 pairs. Expected from MILP: $2^{-8} = 3.906 \times 10^{-3}$. **Ratio: 2.00×.**

The trail exists at r=4. The single-trail probability is a factor 2 higher than the MILP predicts, consistent with the clustering factor documented in §3.3. **The MILP model is correct within a factor of 2 at this round count.**

### 3.3. Empirical check (r=4, w0b0)

As described above, the direct verification gives a ratio of 2.00× between the observed probability and the MILP single-trail prediction. Two interpretations are consistent with the data:

- **Differential clustering** (positive): multiple trails with the same output difference Δout contribute to the observed probability, so the effective probability is higher than the single-trail probability.
- **Model systematic overestimate** (less likely): the AND model `a = Δx ∨ Δy` underestimates the per-round cost by a factor that happens to be 2 at r=4. This would imply the true weight is 9, not 8.

Without a full clustering analysis (summing over all trails with the same Δin, Δout), we cannot distinguish these two. The practical implication is the same: the effective weight of the r=4 trail is 7, not 8. **If this factor holds at higher round counts, the "128-bit threshold at r≈15" becomes "at r≈17–18". The margin remains very large, but the extrapolation should be read as approximate.**

### 3.4. What this section does not establish

- A lower bound on the best single-trail weight (requires gap=0 and multi-bit inputs).
- An exact clustering analysis (requires modeling all trails with the same Δout).
- Resistance to related-key differentials.
- That the factor-2 clustering is constant across all round counts.

## 4. Linear cryptanalysis

### 4.1. Three models, no bound

We tried three linear models. **None produced trails that verified empirically at w=32.** The honest conclusion is that no linear bound is established for MYSX.

#### 4.1.1. Hybrid model (`act = α ∨ β ∨ γ`, no `γ ≥ α`)

This model produced the following single-trail weights:

| r | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| weight | 0 | 2 | 6 | 8 | 10 | 18 | 32 | 48 | 72 | 109 |

Empirical verification targeted the r=4 trail (weight 8, expected correlation $2^{-8} = 0.0039$). Eight independent runs at N=2,000,000 each ($\sigma = 1/\sqrt{N} \approx 7.07\times10^{-4}$) gave a mean observed correlation of $-4\times10^{-4}$, corresponding to $|\text{corr}|/\sigma \approx 0.57\sigma$ per run. If the trail existed with the predicted correlation, it would appear at $|\text{corr}|/\sigma \approx 5.5\sigma$. It does not. **The predicted trail does not exist at w=32.**

A confirmatory experiment at w=8 (mini-MYSX with keyword `'MYSZ'`, non-degenerate) tested α = word 0 bit 0 against 74 single- and multi-bit β masks over 4 independent keys × $2^{19}$ samples each. Observed max $|\text{corr}|/\sigma = 3.83$ at r=2, below the expected maximum of $3.38\sigma$ for 296 independent estimates under $H_0$ (observed/expected ratio 1.13×). No linear signal detected. Consistent with the w=32 result.

Diagnosis: the model allowed `(α,β,γ) = (1,0,0)`, which has correlation 0 in the real AND (not 1/2). The trails found were artifacts.

#### 4.1.2. Strict model (`x = y = z = w` in XOR)

This model returned INFEASIBLE in all rounds 1–10 for mask w0b0. The model is too restrictive: it forbids valid trails with correlation 1 (e.g., the trivial r=1 trail with weight 0).

#### 4.1.3. Relaxed model with correct AND (`x+y+z-w=2u, γ = α OR β`)

This model produced the following single-trail weights:

| r | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| weight | 0 | 2 | 8 | 18 | 35 | 86 | 157 | 244 | 338 | 434 |

Note that these weights differ from those of the hybrid model (§4.1.1), confirming that the two models are not equivalent.

Empirical verification targeted the r=3 trail (weight 8, expected correlation $2^{-8} = 0.0039$). A single run at N=5,000,000 ($\sigma = 1/\sqrt{N} \approx 4.47\times10^{-4}$) gave an observed correlation of $-2.38\times10^{-4}$, at $|\text{corr}|/\sigma \approx 0.53\sigma$. If the trail existed with the predicted correlation, it would appear at $|\text{corr}|/\sigma \approx 8.7\sigma$. It does not. **The predicted trail does not exist at w=32.**

Diagnosis: the model has false positives. The most likely cause is the AND model: the `act = α ∨ β ∨ γ` constraint (and its `γ ≥ α`, `γ ≥ β` refinement) does not capture the shared-variable structure of F, where the two rotated inputs come from the same `x`. A correct linear model of F's correlation requires modeling the rank of the `i-1` to `i-8` connectivity graph, not the per-bit AND activity. We did not implement this model in v0.1.

### 4.2. Why this matters

The best published attacks against Simon128/128 are linear (hull attacks covering ~49/68 rounds, complexity $\sim 2^{127}$). The linear side is therefore the most likely attack surface for MYSX. **A negative result on three models is not a security claim.** It is an open question.

To establish a linear bound for MYSX, future work would need:

1. **Sign tracking** (±1/2 per active AND), which the current MILP does not model.
2. **Linear hull analysis** (clustering of many trails with the same mask).
3. **A SAT/MILP model with XOR correlations < 1**, using the exact F-connectivity structure.

None of these is done here. **MYSX's linear security is unknown.**

## 5. Related-key cryptanalysis

Tool: `keyschedule_test.js` (basic) + `keyschedule_test_extended.js` (22 tests). Coverage: exhaustive weight-1 (512 bits), exhaustive weight-2 (130,816 pairs), weight-3 "same Δ" (1,680 trials), beam search up to weight-6 (beam 50), 5 base keys, structured patterns, cycles, weak keys.

**Results (summary):**

- No persistent $\Delta rk[r]=0$ for $r \geq 1$ in any test.
- Beam search score: 139 (ideal 128).
- No weak keys, cycles, or fixed points.
- Slow diffusion at r=1–4 (expected: one Feistel round per cipher iteration).
- Worst bit (minimum weight at r=1): bit 160 (word 5, bit 0), 1-bit difference. Consistent across 5 base keys (structural, not noise).

**Limitations.** These are heuristic tests, not a proof. We did not run a targeted MILP/SAT related-key model (~650k variables); adaptive related keys, biclique attacks, and algebraic attacks on the key schedule are not covered. **MYSX's related-key security is unknown.**

## 6. Generic attacks

| Attack | Complexity | Verdict |
|---|---|---|
| Classical brute force | $2^{512}$ operations | Infeasible |
| Grover (quantum) | $2^{256}$ iterations | Infeasible with foreseeable technology |
| Block collisions (birthday) | $2^{128}$ blocks | Infeasible |

**Note on Grover.** With a 512-bit key, Grover leaves 256 bits of effective security. With 1,000,000 logical qubits at 1 ns/iteration, the time would be $\sim 10^{60}$ years. This is a

theoretical estimate; no post-quantum analysis of the cipher beyond Grover has been done.

**Note on the 512-bit key.** HKDF-SHA256 produces at most a 256-bit pseudorandom key. If the master secret has 256 bits of entropy, the effective security is 256 bits, not 512. The "key size" of MYSX is not the security level of the system.

## 7. Simon cryptanalysis: what transfers and what does not

MYSX uses Simon's F. It is **not** Simon. The differences:

- Block size and branch organization: MYSX is an SPN-like structure with four parallel Feistel cells over 32-bit word pairs (Section 2.1). Simon is a classical 2-branch Feistel over n-bit halves.
- Four parallel F per round (Simon uses 1).
- Myszkowski byte transposition (Simon has none).
- Round-dependent word rotations (Simon has none).
- Non-linear ARX key schedule (Simon's is linear).
- 128 rounds vs 68.

**What transfers.** The linear and differential distinguishers on Simon's F are properties of the function, not of the key schedule. Any distinguisher on F may in principle be embedded in MYSX's structure. The Simon-specific attacks on round-reduced Simon do not transfer directly because the surrounding state, diffusion, and round count are different, but the F-level biases are the natural starting point for attacking MYSX.

**What does not transfer.** Key recovery from Simon's linear key schedule does not transfer: MYSX's key schedule is non-linear.

**Relevant literature and published attacks on Simon128/128** (68 rounds):

- Linear-hull distinguishers covering ~49 rounds, complexity $\sim 2^{127}$ (Chen–Wang, "Improved linear hull cryptanalysis of round-reduced versions of SIMON", ePrint 2014/973).
- Differential distinguishers covering ~46–50 rounds, complexity $\sim 2^{125.7}$ (Biryukov–Roy–Velichkov, "Differential analysis of block ciphers SIMON and SPECK", CRYPTO 2015).
- SAT-based algebraic distinguishers covering ~12–14 rounds (practical on a PC).
- Integral distinguishers covering ~26 rounds.
- Structural observations on the SIMON family (Kölbl–Leander–Tiessen, "Observations on the SIMON block cipher family", CRYPTO 2015).

None of these results is transferred to MYSX in this paper. The MYSX-specific MILP analysis (Sections 3, 4, 5) is what provides the evidence; Simon attacks are context, not direct transfer.

## 8. Analysis limitations

- **Differential**: upper bounds on single-trail for one input at r=12–14; one trail verified at r=4 (ratio 2.00×); no lower bounds; no full clustering analysis.

- **Linear**: no verified trail in three models; no bound.
- **Related-key**: heuristic tests up to weight-6; no targeted MILP.
- **Integral, impossible, boomerang, algebraic**: not analyzed.
- **No independent public cryptanalysis.**
- **No post-quantum analysis** beyond the trivial Grover bound.
- **No side-channel analysis.**
- **Functional tests** (`mysx.test.js`, 16/16 PASS) cover round-trip, AEAD negatives, nonce policy, PBKDF2, avalanche. They do not prove security.

# 9. Engineering notes

## 9.1. API surface

- Public API: `aeadEncrypt`, `aeadDecrypt`, `deriveAeadKeys`, `deriveAeadKeysFromPassword`, `NonceGuard`.
- Test API: **disabled by default**. Enabled only when the environment variable `MYSX_ENABLE_TEST=1` is set in the process before requiring the module. There is no code-level toggle.
- CTR (`_ctrXor`) is not exported from the public API.

## 9.2. NonceGuard

`NonceGuard` is a best-effort defense against nonce reuse. It:

- counts messages per key (default $2^{32}$);
- detects recent nonce reuse (window of 1024 per key);
- caps the number of tracked keys (default 1024, LRU eviction);
- identifies keys by a non-cryptographic 64-bit hash (FNV-1a variant), **not** by the raw key bytes; `onSave(state)` therefore does not leak the secret key;
- throttles `onSave` calls (default: once every 100 `check()` calls) to avoid per-message state copies;
- **does not persist across restarts by itself**;
- **does not share state across processes or devices**.

Persistence is provided by `onSave(state)` and `initialState` at construction, or by `save()` / `load(state)` for manual control.

**It is not a substitute for key lifecycle management.** If the application reuses `(key, nonce)` across restarts, CTR confidentiality is broken. See `mysx.prod.js` for the exact contract.

## 9.3. aeadEncrypt size validation

`aeadEncrypt` validates plaintext size **before** allocating the CTR output buffer, and enforces a configurable `MAX_PLAINTEXT_BYTES` limit (aligned with `MAX_CT_BYTES`, $2^{31}-1$). Applications that require larger limits should raise both limits explicitly and manage memory accordingly.

### 9.4. Type validation

All public functions validate that byte inputs are `Uint8Array` of the expected length (where applicable). Passing a `string` as `aad` or `plaintext` throws instead of silently coercing to array-like behavior.

### 9.5. KAT digests

Two digests are documented in `README.md`:

- `KAT.json` public digest: SHA-256 over the full JSON (block + AEAD + KDF). This is the artifact a third-party implementation should reproduce.
- `KNOWN_KAT_DIGEST` (internal): SHA-256 over 16 blocks with `key[i] = i`. Used as fast regression in `mysx.test.js`. It is strictly a subset of the public KAT, not a different KAT.

A third-party implementation should target the **public digest**.

### 9.6. Harness validation

The linear sampling harness is validated against a hand-computed case at w=8 (r=1, α = {word0 bit2, word4 bit0}, β = word0 bit3, fixed key). The linear approximation $x_0 \oplus F(x)_2 = x_1 x_6$ has correlation exactly ±1/2 with fixed key. Observed correlation: **0.4993** (expected 0.5, within sampling noise). With random keys per sample, the sign of the correlation flips per sample and the observed |corr| drops to **0.0022** (expected ≈ 0).

This confirms the harness is correct when used with fixed keys. It also confirms that the practice of "random keys per sample", used in an earlier version of `mini_trail.js`, washes out genuine linear signals and must not be used for linear cryptanalysis.

See `validate_linear_harness.js`.

The mini-MYSX with `w=8, kw='MYSZKOWS'` is **degenerate**: the permutation becomes `[4,0,5,2,7,6,1,3]`, the L→R→L cycle at word 0 means a single-bit difference never passes through F, and the differential probability is 1 at all rounds. This is not representative of MYSX. Use `w=8, kw='MYSZ'` (non-degenerate from r=3) or w=32 directly. See `check_mini_nondegenerate.js`.

## 10. Verdict

**MYSX-256/512 v0.1-draft is an experimental proposal, not a secure cipher.** The differential analysis reports upper bounds on single-trail weights, not a security proof; one trail at r=4 has been directly verified with clustering factor 2.00×. The linear analysis is a negative result on three models, not a bound. The related-key analysis is heuristic. No integral, impossible-differential, boomerang, or algebraic analysis has been done. The cipher has not been externally audited.

**Do not use MYSX to protect real data.** For production: **AES-256-GCM** or **XChaCha20-Poly1305**.

## 11. Reproducibility

| File | Purpose |
|---|---|
| `SPEC.md` | Formal specification (identical content to Section 2) |
| `PAPER.md` | This document |
| `KAT.json` | Test vectors (block, AEAD, KDF) |
| `generate_kat.py` | KAT generator |
| `mysx.prod.js` | Implementation (public API) |
| `mysx.test.js` | Functional tests |
| `mysx_milp.py` | Differential MILP |
| `mysx_linear_milp.py` | Linear MILP (three models) |
| `keyschedule_test.js` | Basic key schedule audit |
| `keyschedule_test_extended.js` | Exhaustive audit (22 tests) |
| `check_trail.js` | Empirical differential trail at r=4 |
| `check_linear_trail2.js` | Empirical linear trail at r=3 |
| `validate_linear_harness.js` | Hand-computed linear harness check |
| `mini.js` | Mini-MYSX (w=8, w=16, w=32) |
| `check_mini.js` | Mini-MYSX validation vs `mysx.prod.js` |
| `check_mini_nondegenerate.js` | Keyword/w compatibility check |
| `mini_linear_fast.js` | Linear sampling at w=8, `kw='MYSZ'` |
| `verify_trail_w32.js` | Per-trail verification at w=32 with MILP weight |
| `avalanche.js` | Diffusion and avalanche ladder (r=1..24) |

Fixed seed in tests (`0xC0FFEE`). MILPs are deterministic given the solver and `gapRel`. The mini-MYSX with `w=32` reproduces `mysx.prod.js` exactly (F function and MYZ_PERM are bit-identical, verified in `check_mini.js`).

## 12. Final warning

**MYSX has not been externally audited.** This paper is the author's self-analysis, not a certification. No result here replaces an independent cryptographic review.

**Do not use MYSX to protect real data.**